Mini-Project Report: ArUco Station Identification & Task Triggering

Student Name: Amin Ahmed G | Dept: B.E. Robotics & Automation Engineering, Anna University
Package: secure_station_dispatch (Phase 1 Deliverable) | Environment: ROS 2 Jazzy (Ubuntu 24.04)

1. Title & Objective

The main goal of this mini-project is to build a simulated AMR dispatch pipeline using ROS 2. We place 4×4 ArUco markers at four industrial stations (Inspection, Assembly, Storage, and Charging). When the robot's camera detects a marker, the system identifies which station it reached and triggers a simulated multi-step task using ROS 2 action interfaces and a finite state machine.

2. Tech Stack

• ROS 2 Distribution: ROS 2 Jazzy Jalisco (Python 3.12)

• Computer Vision: OpenCV (cv2.aruco module) and cv_bridge

• Communication: ROS 2 Topics (Publishers/Subscribers), ROS 2 Actions (Client/Server)

• Custom Interfaces: StationMarker.msg and StationTask.action

• Configuration & Logging: YAML (station_registry.yaml), JSON Lines (dispatch_events.jsonl)

• Build System: colcon with CMake (CMakeLists.txt) and Python setup.py

3. System Architecture

I split the system into three separate nodes mainly so I could test and debug each part—vision processing, state control, and task execution—on its own. It also mirrors how a real AMR separates perception, decision-making, and execution into distinct modular packages rather than running everything inside a single monolithic node.

1. marker_detector_node.py: Listens to /camera/image_raw, processes the frame with OpenCV's DICT_4X4_50 ArUco detector, and publishes a StationMarker message on /station_marker whenever it sees a marker.

2. dispatcher_node.py: Runs a simple two-state machine (IDLE and TASK_IN_PROGRESS). When it's in IDLE and gets a valid marker message, it switches to TASK_IN_PROGRESS, logs the event, and sends an action goal to the task server.

3. task_simulator_node.py: Serves as the action server for /station_task. It receives goals from the dispatcher, runs through a 5-step simulated work sequence with periodic progress feedback, and returns success when done.

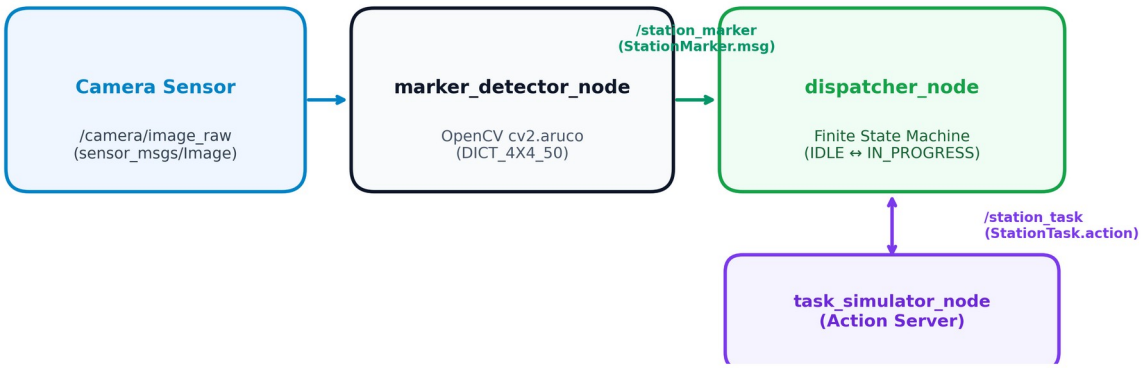


Figure 1: 3-Node System Architecture Flow Diagram

4. Implementation Details

For custom interfaces, StationMarker.msg passes the detected marker ID, station name, estimated pose, confidence score, and timestamp:

```
int32 marker_id
string station_name
float64[7] robot_pose
float64 confidence
builtin_interfaces/Time stamp
```

StationTask.action handles the action goal, final result summary, and step-by-step progress feedback:

```
# Goal
string task_type
string station_name
---
# Result
bool success
string summary
float64 execution_time
---
# Feedback
string status
float64 progress
```

The mapping between marker IDs and station tasks is defined in station_registry.yaml:

Marker ID	Station Name	Task Type	Sim Duration
0	Inspection Station	Inspection	~3.0 s
1	Assembly Station	Assembly	~3.5 s

2	Storage Station	Storage	~2.7 s
3	Charging Station	Charging	~3.5 s

The dispatcher state machine starts in IDLE. Once a marker message comes in, it transitions to TASK_IN_PROGRESS and ignores any new marker detections so it won't interrupt the ongoing task. When the action server returns the final result, the dispatcher writes to dispatch_events.jsonl and drops back to IDLE.

5. Testing & Results

I wrote an integration script (test_dispatch_pipeline.py) that tests all four markers sequentially. Here is the console output from the test run:

```
============================================================
  PIPELINE TEST SUMMARY
============================================================
  Events logged    : 8
  DISPATCHED events: 4
  COMPLETED events : 4

  Marker 0: dispatched=True  completed=True   ✓ PASS
  Marker 1: dispatched=True  completed=True   ✓ PASS
  Marker 2: dispatched=True  completed=True   ✓ PASS
  Marker 3: dispatched=True  completed=True   ✓ PASS

  OVERALL: ✓ PASS — Phase 1 pipeline working
============================================================
```

During testing, the dispatcher creates logs/dispatch_events.jsonl and appends JSON records for every dispatch and completion event:

```
{"timestamp": "2026-08-18T07:00:01.123+00:00", "ros_time": 1234567890000,
"marker_id": 0, "station": "Inspection Station", "outcome": "DISPATCHED",
"summary": ""}
{"timestamp": "2026-08-18T07:00:04.456+00:00", "ros_time": 1234570890000,
"marker_id": 0, "station": "Inspection Station", "outcome": "COMPLETED",
"summary": "Inspection Station complete at Inspection Station in 3.0 s (5
steps)"}
```

6. Challenges Faced

The main issue I ran into while building this was a CMake target name collision during colcon build.

In standard ROS 2 Python packages, CMakeLists.txt uses ament_python_install_package(secure_station_dispatch). But because I also defined custom

message and action interfaces (StationMarker.msg and StationTask.action) in the same package using rosidl_generate_interfaces(), ROS 2 Jazzy generated internal CMake target names that conflicted with ament_python_install_package(). The build kept failing with target redefinition errors.

To fix it, I removed ament_python_install_package() from CMakeLists.txt and used install(DIRECTORY) to place the Python files directly into site-packages:

```
install(DIRECTORY secure_station_dispatch/
  DESTINATION lib/python3.12/site-packages/secure_station_dispatch
)
```

After changing this, the package built cleanly with colcon build and both the custom interface headers and Python modules installed correctly.

7. Conclusion

The Phase 1 pipeline works as expected. The marker detector extracts ArUco IDs from camera images, the state machine handles state changes without getting stuck or double-dispatching, and ROS 2 actions handle task execution with step feedback. All dispatch events get logged properly to a JSON file.